What is a software requirement?

Requirements Engineering (RE) is the systematic process of discovering, analyzing, documenting, validating, and managing the requirements of a software system. **Before writing code, software engineers must understand exactly what users need. This article discusses the 2011 revisions to the IEEE 830 standard for requirements analysis**.

What are the characteristics of good software requirements?

The IEEE has a standard, **IEEE 830** that lays out the characteristics according to the IEEE of good software requirements:

Characteristic	Definition	Why It Is Important
1. Correct	The Software Requirements Specification (SRS) should accurately describe the expected system behavior and user needs. It should not include unrealistic or incorrect requirements.	Ensures the software solves the intended problem and meets user expectations.
2. Unambiguous	Requirements should be written clearly so that every reader interprets them in the same way.	Prevents misunderstandings between customers, developers, and testers.
3. Complete	The SRS should include all required system behaviors, features, constraints, and interactions.	Ensures no essential functionality is omitted during development.
4. Consistent	Requirements should not contradict one another. All requirements should be logically compatible.	Prevents confusion and conflict implementation during development.
5. Ranked (Prioritized)	Requirements should be prioritized according to their importance, urgency, or business value.	Helps developers implement the most critical features first and manage limited resources effectively.
6. Verifiable	Every requirement should be measurable and testable so that it can be verified after implementation.	Enables testers to confirm whether the software satisfies each requirement.
7. Modifiable	The SRS should be organized so that requirements can be easily updated or changed without introducing inconsistencies.	Simplifies maintenance when requirements evolve during the project lifecycle.
8. Traceable	Each requirement should have a unique identifier and be traceable throughout design, implementation, testing, and maintenance.	Helps track the origin, implementation, and verification of every requirement throughout the software development process.

Example: Good Software Requirements with Library Management System

Characteristic	Poor Requirement	Good Requirement
1. Correct	The system shall allow students to borrow unlimited books without any restriction. *(Contradicts university policy.)*	The system shall allow each student to borrow a maximum of 5 books at a time according to university library policy.
2. Unambiguous	The system should be fast.	The system shall display book search results within 2 seconds under normal operating conditions.
3. Complete	The system shall manage books.	The system shall allow librarians to add, update, delete, search, issue, return, and generate reports for books.
4. Consistent	Requirement 1: Students may borrow 5 books. Requirement 2: Students may borrow 10 books.	All requirements consistently specify that each student may borrow up to 5 books.
5. Ranked (Prioritized)	All requirements are treated equally.	High Priority: Login, Search Books, Borrow Books. Medium Priority: Email Notifications. Low Priority: Dark Mode Interface.
6. Verifiable (Testable)	The interface should be user-friendly.	At least 90% of users successfully search and borrow a book without assistance during usability testing.
7. Modifiable	Borrowing rules are hard coded in multiple modules.	The maximum borrowing limit shall be stored in a configuration file or database so it can be changed without modifying the source code.
8. Traceable	The requirement has no identifier or source.	Requirement ID: FR-03. Source: Interview with Head Librarian (15 August 2026). Related Design Module: Book Borrowing Module. Test Case: TC-03.

What are some wording and language best practices for software requirements?

Best Practice	Description	Bad Example	Good Example
Use "shall" for mandatory requirements	Use **shall** to specify mandatory system behavior.	The system **should** allow login.	The system **shall** allow users to log in using a username and password.
Use clear and simple language	Avoid complicated or confusing sentences.	The application should facilitate authentication procedures.	The system shall allow users to log in.
Avoid ambiguity	Use precise and measurable words instead of vague terms.	The system shall be fast.	The system shall display search results within **2 seconds**.
Write one requirement at a time	Each requirement should describe only one function or constraint.	The system shall allow login and generate reports.	FR-01: The system shall allow users to log in. FR-02: The system shall generate monthly reports.
Make requirements testable	Requirements should be measurable and verifiable.	The interface shall be user-friendly.	At least **90% of users** shall complete the login process without assistance.
Use active voice	Clearly specify who performs the action.	Book information will be updated.	The system shall update book information after the librarian saves the changes.
Be consistent	Use the same terminology throughout the document.	Book, document, item (used interchangeably).	Use **Book** consistently throughout the SRS.
Avoid negative statements	Describe what the system **shall do**, not what it shall not do, unless necessary.	The system shall not fail frequently.	The system should maintain **99.9% availability**.
Assign a unique identifier	Number each requirement for traceability.	No identifier.	**FR-01:** The system shall allow users to log in.
Keep requirements concise	Avoid unnecessary details and long paragraphs.	A long paragraph describing multiple features.	One clear requirement per statement.

Common Words Used in Software Requirements

Word	Meaning	Example
Shall	Mandatory requirement	The system shall store user information securely.
Should	Recommended but optional	The system should provide a dark mode.
May	Optional feature	Users may upload profile pictures.
Must	Mandatory constraint (often used for regulations or safety)	Passwords must contain at least 8 characters.
Can	Describes capability	Administrators can generate reports.

Note: In formal SRS documents, **"shall"** is preferred over **"must"** for functional requirements because it clearly indicates a mandatory requirement.

Words to Avoid

Avoid vague words because they cannot be tested.

Avoid	Better Alternative
Fast	Within 2 seconds
Easy	Complete the task in 3 steps or fewer
User-friendly	90% of users complete the task without assistance
Efficient	Process 100 transactions per minute
Secure	Encrypt passwords using AES-256
Quickly	Within 5 seconds

Best Rules:

- Use **shall** for mandatory requirements.
- Write **one requirement per statement**.
- Make requirements **specific and measurable**.
- Avoid **ambiguous words** (e.g., *fast*, *easy*, *good*).
- Use **consistent terminology** throughout the SRS.
- Assign a **unique ID** (e.g., FR-01, NFR-01).
- Ensure every requirement is **testable** and **traceable**.

Flexible	Specify exactly what can be configured
Appropriate	Describe the exact expected behavior

How do you rank software requirements?

Definition: Ranking (or prioritization) is the process of assigning an importance level to each software requirement. This helps the development team decide **which features must be implemented first** when there are constraints such as limited time, budget, or resources.

Three-Level Requirement Ranking

Priority Level	Description	Example (Library Management System)
Mandatory (Must Have)	Essential requirements. The system cannot function without them.	User login, Search books, borrow books, Return books, Store borrower information.
Desirable (Should Have)	Important features that improve the system can be postponed if necessary.	Email reminders for due dates, Book reservation, Fine calculation, Barcode scanning.
Optional (Nice to Have)	Additional features that enhance user experience but are not essential.	Dark mode, Personalized book recommendations, Reading statistics dashboard, Book reviews and ratings.

Example Scenario

Suppose the university gives your team:

- Budget: ৳10,00,000
- Time: 3 months
- Development Team: 4 developers

Initially, the customer requests the following features:

Requirement	**Priority**
User Login	Mandatory
Search Books	Mandatory
Borrow Books	Mandatory
Return Books	Mandatory
Fine Calculation	Desirable
Email Notifications	Desirable
Book Reservation	Desirable
Dark Mode	Optional
Reading Statistics	Optional

Situation

After two months, the project is behind schedule.
The development team decides:

- Complete all Mandatory requirements.
- Implement Fine Calculation if time permits.
- Postpone Dark Mode, Reading Statistics, and Book Rating System to a future version.

The final product is delivered on time while still meeting the users' essential needs.

Book Rating System	Optional

Priority	Action	Examples
Mandatory	Must be implemented	Login, Search, Borrow, Return
Desirable	Implement if resources permit	Fine Calculation, Email Notification, Book Reservation
Optional	Future enhancement	Dark Mode, Book Reviews, Reading Statistics

What is the role of the requirements document in the software development process?

Definition: A Requirements Document (also called the Software Requirements Specification (SRS)) is the official document that describes what the software system should do and the constraints under which it must operate.

Important: An SRS describes **"What"** the system should do, **not "How"** it will be implemented.

Roles of the Requirements Document

Role	Description	Library Management System Example
Defines System Requirements	Clearly describe what the software should do.	The system shall allow students to search books by title, author, or ISBN.
Acts as an Agreement (Contract)	Serves as a common understanding between the customer and the development team.	The university and the software company agree on the listed features before development begins.
Guides System Design	Provides designers with the information needed to create	Since the SRS requires book borrowing, designers

	software architecture and database.	create the Borrow Book module.
Guides Software Development	Developers use the SRS as the primary reference during implementation.	Developers implement login, search, borrow, and return features exactly as specified.
Supports Software Testing	Test engineers create test cases to verify that each requirement has been implemented correctly.	Test Case: Verify that a student cannot borrow more than five books.
Helps Validate Customer Needs	Ensures that the documented requirements match stakeholder expectations before development begins.	Librarians review the SRS and confirm that all required library functions are included.
Supports Change Management	Makes it easier to update and track changes when requirements evolve.	A new requirement for QR-code book checkout is added and documented as a new requirement.
Provides Project Documentation	Serves as a permanent reference throughout development, maintenance, and future upgrades.	Future developers can understand the original system requirements by reading the SRS.

How is the Software Requirements Specification (SRS) used throughout the Software Development Life Cycle (SDLC)?

SDLC Phase	Role of the SRS	Example (Library Management System)
Requirements Document (SRS)	Define what the system should do. It serves as the official agreement between stakeholders and the development team.	Requirement: "The system shall allow students to search for books by title, author, or ISBN."
System Design	Designers use the SRS to create the system architecture, database schema, user interface, and UML diagrams.	Design a Book Search Module, database tables (Books, Students), and a search interface based on the SRS requirement.
Implementation	Developers write the source code according to the requirements specified in the SRS.	Implement the search functionality that retrieves books using title, author, or ISBN.

Testing	Testers prepare test cases to verify that every requirement in the SRS has been correctly implemented.	Test Case: Search for a book by ISBN. Expected Result: The correct book information is displayed within 2 seconds.
Maintenance	The SRS is used to understand existing functionality when fixing bugs, updating features, or adding new requirements.	The university requests a new feature to search books by publisher. The SRS is updated, and developers modify the search module accordingly.

Who typically uses software requirements documents?

Stakeholder	Role	How They Use the SRS	Library Management System Example
Customers / Clients	Define business needs and approve requirements.	Ensure the SRS reflects their expectations and business goals.	The University Library reviews the SRS to confirm it includes features such as book search, borrowing, returning, and fine calculation.
Developers / Programmers	Implement the software.	Use the SRS as a guide to write code that satisfies the specified requirements.	A developer implements the Book Search feature because the SRS states that students shall search books by title, author, or ISBN.

QA Personnel / Testers	Verify and validate the software.	Develop test cases based on the requirements in the SRS.	A tester verifies that a student cannot borrow more than 5 books, as specified in the SRS.
Project Managers / Team Leaders	Plan, monitor, and control the project.	Estimate cost, schedule, resources, and track project progress using the SRS.	The project manager estimates that the Borrow Book Module will require two developers and two weeks to complete.
System & Maintenance Engineers	Maintain and enhance the software after deployment.	Use the SRS to understand the original system requirements before making changes or fixing defects.	When the university requests QR code-based book checkout, the maintenance team updates the system based on the existing SRS.